



SURGE

Contracts Audit

Yaroslav Tumanov, Eugene Panasenko

February, 2026

1. Introduction. **SURGE** requested 4irelabs to review its contracts.

There are 12 contracts in the scope of this audit. No hash-commit, the files were taken from ["bonding-audit-package.tar.gz"](#).

- MainToken.sol
- MainFactory.sol
- FFactory.sol
- FRouter.sol
- FPair.sol
- Bonding.sol
- BondingETHRouter.sol
- IV2LockerFactory.sol
- FERC20.sol
- AeroAdaptor.sol
- AeroFactoryAdaptor.sol
- AeroRouterAdaptor.sol

2. Warranty. Audit is provided on an "as is" basis, without warranty of any kind, express or implied. The Auditor does not guarantee that the Code Review will identify all instances of security vulnerabilities or other related issues.

3. Executive Summary. This audit covers a streamlined fork of the Virtuals Protocol, focused exclusively on the token bonding and liquidity provisioning components. The project removes the DAO and NFT subsystems, retaining only the smart contracts that handle token creation, bonding-based price discovery, and decentralized trading. The system enables creators to deploy new ERC20 tokens through a factory contract, establish an initial bonding curve for price formation, and transition seamlessly into open-market trading via an integrated automated market maker (AMM).

The AMM LP tokens are locked forever. Depending on the network settings, it could be Aerodrome if Base or burned to a dead address.

The audit focused on validating deployment integrity, initialization parameters, price evolution (initial, pre-graduation, and post-graduation),

liquidity accumulation in the base asset, and token unwrapping mechanisms. Different flows were tested to ensure consistent state transitions, mathematical correctness of bonding formulas, and safe interaction between factories, pairs, and routers.

Inherent / Unavoidable Risks

Certain risks are intrinsic to bonding-curve and virtual supply mechanisms. These arise not from implementation vulnerabilities or bugs but from fundamental trade-offs in the economic design. The following risks cannot be completely eliminated without altering the core behavior of the protocol:

A) Economic Imbalance Between Early and Late Participants

Bonding curves, by nature, reward early entrants and penalize later ones as the price rises. Adjusting slope or initial supply can smooth this effect, but never remove it without losing the intended “progressive price discovery” mechanism.

B) Liquidity Shock During Large-Scale Redemptions or Graduations

Any system allowing users to unwrap or redeem tokens will risk sudden supply expansions or contractions. Throttling and cooldowns can moderate this, yet the volatility itself is an unavoidable side-effect of freely tradable bonded assets.

C) Market-Dependent Asset Valuation

The protocol’s token price depends on external liquidity and speculative demand. No contract-level change can stabilize this

D) Graduation Threshold Sensitivity

The point at which bonding transitions to “graduation” (or supply unlock) introduces discontinuity in price and supply. Even with

smoothing mechanisms, it inherently represents a regime shift that may trigger short-term volatility.

4. **Critical Bugs and Vulnerabilities.** No Critical bugs and vulnerabilities have been found in the **SURGE** token contracts. All reported issues are low or informal and don't require any urgent fixes.
5. **Initial parameters review and comparison with the code base.**

Parameter	Meaning	Comment & Risk Evaluation
proposalThreshold = 50000	Minimum tokens needed to create governance proposals.	✅ Low risk. Since you've removed DAO logic, this variable has no functional effect. Harmless unless reused in the future.
agentTokenSupply = 1,000,000,000	Total minted supply of the Agent token.	✅ Good supply. Large supply numbers are mostly cosmetic but may affect user psychology. 18 decimals insured.
agentTokenLpSupply = 200,000,000	Portion allocated to liquidity pool.	✅ Good ratio. Keeping about 20% in LP is reasonable for price discovery and market making. Too little LP leads to high slippage; too much LP dilutes reserve effects.

Parameter	Meaning	Comment & Risk Evaluation
agentTokenVaultSupply = 0	Tokens held for treasury/rewards.	✅ Good design choice. No vault allocation means no budget for incentives, refunds, or gradual emissions. But, at the same time, there's no hidden stash of tokens under the deployer. Every token comes from bonding, so no one can later "dump"
botProtection = 0	Whether anti-sniping / anti-MEV protection is active.	⚠️ Acceptable for testing, risky in production. MEV bots can front-run early buys/sells during bonding or graduation. You might add temporary transaction throttling or block-time-based limits if deployed publicly.
agentTax = 0	Transfer tax applied to Agent token trades.	✅ User-friendly design choice. No tax improves UX and volume, but removes a passive income or burn mechanism that could stabilize price.

Parameter	Meaning	Comment & Risk Evaluation
swapThreshold = 100	Minimum number of tokens before swapping accumulated fees.	✅ Mostly irrelevant if agentTax = 0 (no fees accumulate). Safe default, but could be reduced for smaller-scale deployments.
bondingFee = 1	Only charged once, during the initial launch. Fixed fee of 0.001 base token.	✅ Healthy value. Low enough not to deter users, high enough to prevent spam. Just make sure project docs clearly say "fee is charged once per bonding curve deployment, not per trade."
bondingInitialSupply = 1,000,000,000	Number of bonding tokens initially created.	✅ Ultra-fair – full supply (1B tokens) on the curve, no pre-mints/VC dumps; anyone can buy at a dynamic price.
bondingAssetRate = 28,000,000	Conversion rate: how much base asset per bonding token (initial slope).	✅ Excellent default. This heavily influences price progression. A small miscalibration can make the curve too steep (price explodes) or too flat (no incentive). Test empirically in a simulation.

Parameter	Meaning	Comment & Risk Evaluation
bondingMaxTx = 100	Maximum % of token amount per bonding transaction.	⚠️ Risky in production. Pros: Unlimited transfers, nobody hits weird “max transaction” reverts. Pure permissionless—anyone grabs value fast; viral pumps possible. Cons: Whale domination/manipulation risk
bondingGradThreshold = 200,000,000	Supply threshold at which bonding “graduates” (stops minting new tokens).	✅ Strong default – balanced industry standard. If set too high, price discovery ends prematurely; too low, and users lose interest before graduation events occur.

6. Price flow simulation

A series of bonding curve simulations was conducted to observe the system's pricing behavior under different participant profiles. No anomalies or unexpected results were identified.

It is additionally noted that following the graduation event, the token price on Uniswap experiences an approximate 20% correction, which aligns with expected market rebalancing behavior and is therefore not considered an issue.

Figure 1 illustrates a normal scenario where approximately 100 regular users purchase incrementally, resulting in a gradual and steady price increase.

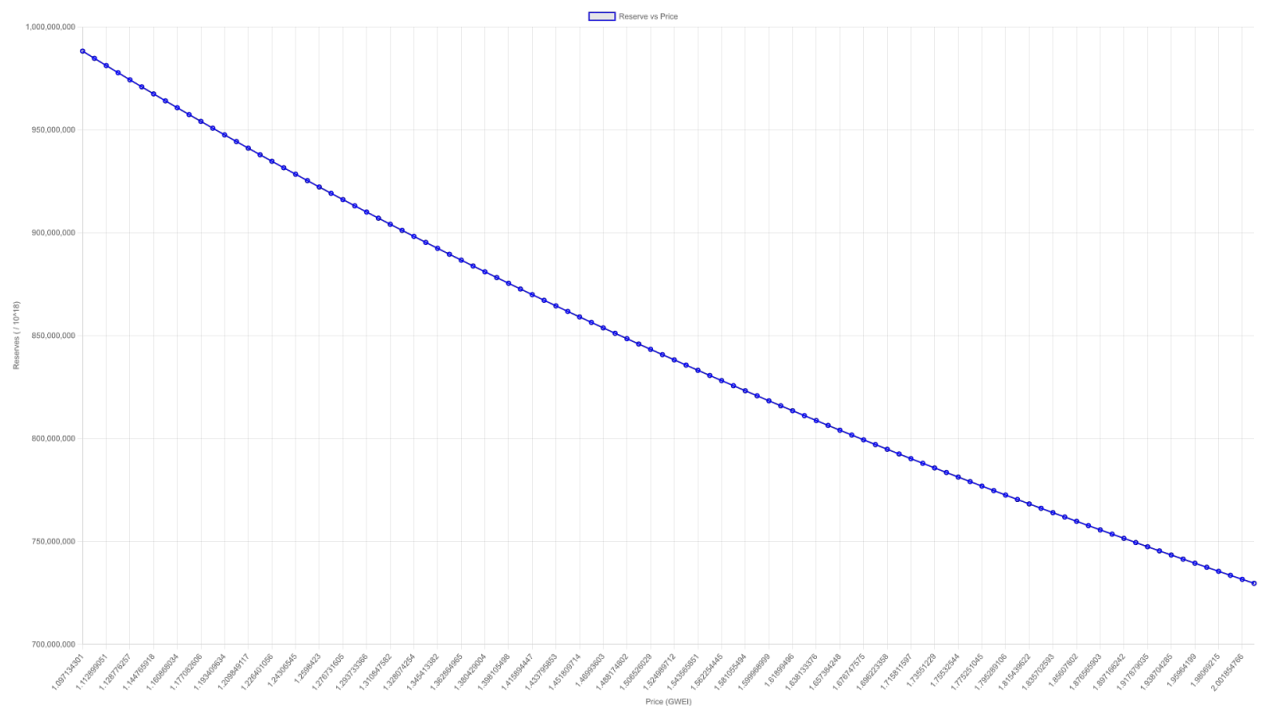


Figure 2 demonstrates the impact of 10 large (“whale”) participants making substantial purchases, causing a pronounced and rapid price rise.

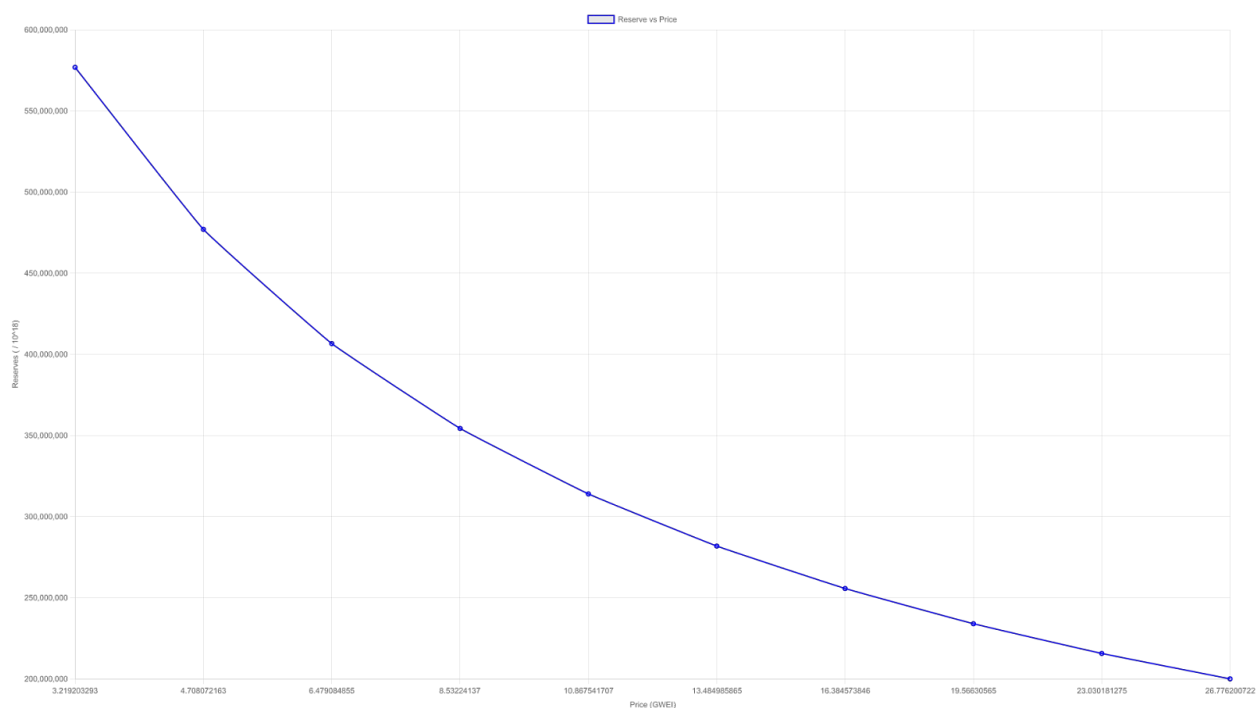
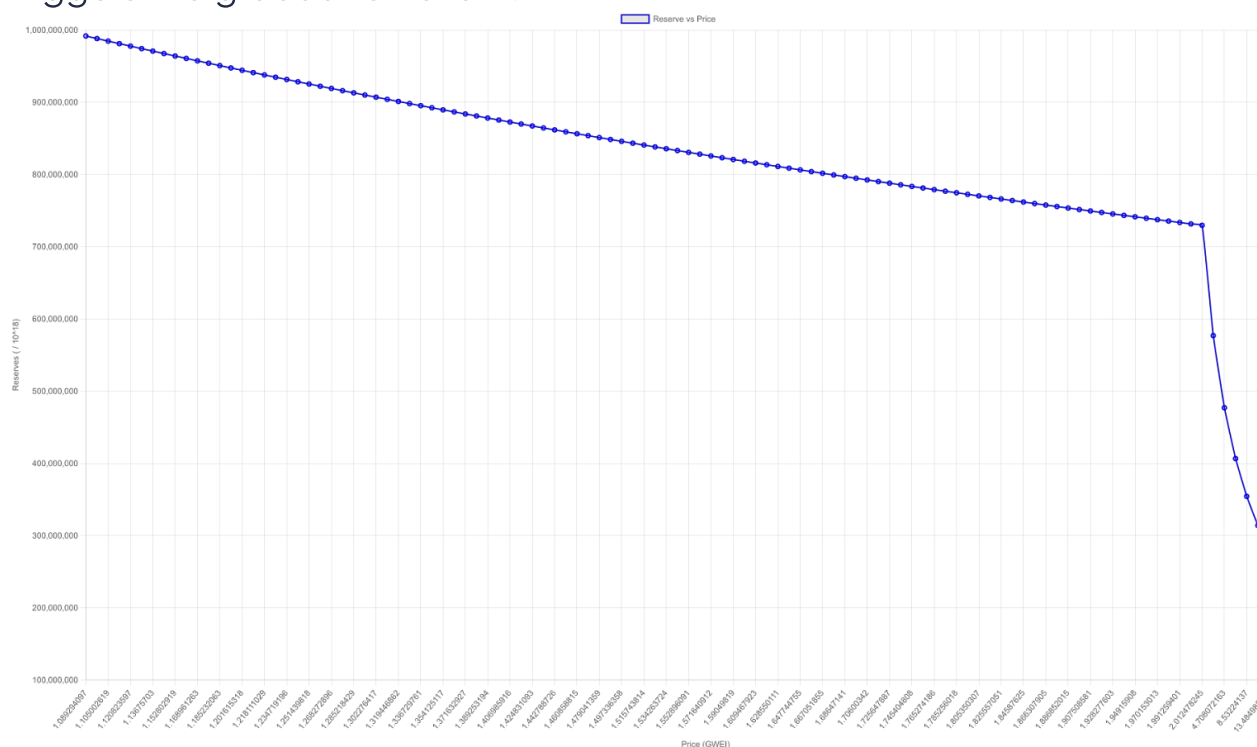


Figure 3 combines both scenarios, showing how the interaction of regular buyers and whale participants accelerates the price progression and triggers the graduation event.



7. Testing and verification

Unit tests were executed using Mocha/Chai on Hardhat.

The test suite confirms successful deployment, initialization, and expected core contracts behaviors.

Aerodrom's lock and the adapters were tested on a fork.

All tests passed successfully, indicating that contract deployment and main flows function as designed.

8. Line By Line Review.

8.1. MainFactory.sol:

8.1.1. Lines 17, 18. Low: The contract currently mixes upgradeable and non-upgradeable OpenZeppelin base contracts:

PausableUpgradeable (upgradeable) is used together with AccessControl (non-upgradeable). This pattern is unsafe for proxy/initializer-based deployments, it can break initialization, lead to uninitialized parent state, or cause storage-layout incompatibilities across upgrades. The recommended fix is to use the upgradeable variants for all OpenZeppelin contracts and call each parent initializer explicitly in the implementation initialize(...) function.

8.1.2. Line 182. Information: Significant centralization risk—the WITHDRAW_ROLE holder can withdraw stakes from any active application (not just their own), potentially draining funds before proposers claim them. While useful for emergency recovery or treasury management, it should be clearly documented and restricted, protected with multi-sig wallet or time-lock.

8.1.3. Multiple lines. Information: setVault, setApplicationThreshold, setParams, totalAgents, pause, unpause, setLockerConfig, lockerFactory, lockerOwner, feeBeneficiary, beneficiaryShare and several other functions marked as public, but no internal calls detected. Consider marking them external for slightly higher gas efficiency.

8.1.4. Lines 257. Information: In _createNewMainToken function we noticed a comment: *tokenOwner (creator) becomes token owner instead of _tokenAdmin.*

Recommendation:

Since _tokenAdmin is not used anymore, it could be safely removed.

8.2. MainToken.sol:

8.2.1. Line 697. Low: In AgentToken::_swapTax, the Uniswap router call to swapExactTokensForTokensSupportingFeeOnTransferTokens uses a hardcoded amountOutMin = 0, permitting unlimited slippage during automatic tax swaps triggered by transfers. This may cause the projectTaxRecipient to receive fewer tokens than expected. To fix, please compute amountOutMin via getAmountsOut with a slippage tolerance (e.g., 3% buffer) and enforce it in the swap, ensuring reverts on unfavorable outputs.

8.2.2. Line 25, 26. Information: Unused constants: ROUND_DEC, CALL_GAS_LIMIT.

8.2.3. Line 227, 232. Information: _uniswapRouter.factory() could be replaced with factoryAddr already initialized at line 224.

8.3. Bonding.sol:

8.3.1. Line 402. Low: Integer Division Truncation in

Bonding._openTradingOnUniswap Leads to precision loss when passing supply and tokenBalance to MainFactory.executeBondingCurveApplicationSalt. May result in a lower-than-intended lpSupply and higher vaultSupply after re-multiplication by 10^{18} in AgentToken.initialize. To fix, pass the full wei values without division, ensuring precise allocation and preventing truncation-induced losses.

8.3.2. Lines 104 and 105. Information: Potentially unsafe deployment of contracts/fun/Bonding.sol:Bonding

Incorrect order of parent initializer calls.

- Found initializer calls to parent contracts in the following order: OwnableUpgradeable, ReentrancyGuardUpgradeable
 - Expected: ReentrancyGuardUpgradeable, OwnableUpgradeable
- Call parent initializers in linearized order.

While technically it violates OpenZeppelin's initialization-order convention, it's not a real vulnerability—no corruption, no privilege escalation, nothing breaks. So, we marked this issue as "information".

8.3.3. Lines 142 and 173. Information: functions **setApprovedRouter**, **launchFor** and **launch** could be marked as external since no internal calls found

8.3.4. Line 278. Information: a minor refactor is possible to avoid code duplication, see

```
if (profile[creator].user == creator) {  
    profile[creator].tokens.push(address(token));  
} else {  
    profile[creator].user = creator;  
    profile[creator].tokens.push(address(token));  
}
```

could be replaced with:

```
profile[creator].tokens.push(address(token));  
  
if (profile[creator].user != creator) {  
    profile[creator].user = creator;  
}
```

8.4. FPair.sol:

8.4.1. Line 21. Information: most functions (getReserves, kLast, priceALast, priceBLast, balance, assetBalance, etc..) are marked public, though they are not called internally.

Impact:

Minor; slightly higher gas cost on external calls and unnecessary internal wrappers.

Recommendation:

Consider marking them external for marginal efficiency gains.

8.4.2. Lines 13, 14, 15. Information: The variables `router`, `tokenA`, and `tokenB` are assigned only once and never modified. Consider declaring them as immutable to reduce gas costs and make the code's intent clearer.

8.4.3. Line 25. Information: The contract inherits from `ReentrancyGuard` but does not use the `nonReentrant` modifier in any function.

Impact:

The inheritance currently has no effect. This may indicate leftover boilerplate or a missed attempt to protect functions involving token transfers.

Recommendation:

Unless reentrancy protection is needed, remove the inheritance to reduce bytecode size and improve clarity.

If protection was intended for token transfer functions (e.g., `swap`, `transferAsset`, or `transferTo`), apply the `nonReentrant` modifier appropriately.

8.4.4. Line 25. Information: `priceALast` and `priceBLast` functions: precision loss for one function and always zero value for the other one.

Recommendation:

add a 10^{18} multiplier or add a comment that briefly explains why the behaviour of the functions is expected.

8.5. FRouter.sol:

8.5.1. Line 102, 148. Low: The `buy` and `sell` functions in `FRouter` do not include slippage protection, allowing trades to execute with

potentially unfavorable amountOut due to price volatility or low liquidity. This exposes users to significant losses from front-running or manipulation, as there's no minAmountOut check to revert if the received tokens fall below an expected threshold. To fix, add a minAmountOut parameter to both functions and require the calculated amountOut $\geq$ minAmountOut before proceeding, similar to standard AMM routers.

8.5.2. Multiple lines. Information: These functions: addInitialLiquidity, sell, buy, graduate, approval, setTaxManager are marked public, though they are not called internally.

Impact:

Minor; slightly higher gas cost on external calls and unnecessary internal wrappers.

Recommendation:

Consider marking them external for marginal efficiency gains.

8.6. FFactory.sol:

8.6.1. Multiple lines. Information: These functions: setRouter, setTaxParams, allPairsLength, getPair are marked public, though they are not called internally.

Impact:

Minor; slightly higher gas cost on external calls and unnecessary internal wrappers.

Recommendation:

Consider marking them external for marginal efficiency gains.

8.6.2. Line 32. Information: Unnecessary nonReentrant Modifier in FFactory.createPair (Gas Optimization). The nonReentrant

modifier on `createPair` is redundant, as the function performs no external calls or state mutations vulnerable to reentrancy. Since that is the only use of reentrancy protection, the `ReentrancyGuardUpgradeable` inheritance becomes unnecessary. If future updates to `_createPair` introduce external interactions, re-add the guard.

- 8.6.3. Information:** It's a good practice to include `uint256[50] private __gap;` at the end of each upgradeable contract (including `FFactory` and other upgradeable contracts in this scope) to future-proof layout changes, especially along inheritance chains. But, it is not a requirement; you don't need a `__gap` to remain upgrade-safe as long as you only ever append new storage variables at the end.

Optional recommendation:

Append a storage gap at the end of the contract to preserve upgrade safety.

8.7. FERC20.sol:

- 8.7.1. Lines 137 and 141. Information:** contract has 2 admin functions: **`updateMaxTx`** and **`excludeFromMaxTx`**. With the current project flow, these 2 functions will not be called, since the owner of the contract is `Bonding.sol`. And, even if `FERC20` is deployed separately, calling those functions is pointless since actual verification is commented:

```
// if (!isExcludedFromMaxTx[from]) {  
  
//     require(amount <= _maxTxAmount, "Exceeds MaxTx");  
  
// }
```

Please consider removing them unless required. If required, please uncomment the section. Also, **updateMaxTx** and **excludeFromMaxTx** functions could be marked as external.

8.8. BondingETHRouter.sol:

8.8.1. Lines 134 and 168. Information: return values of `bonding.buy(...)` and `bonding.sell(...)` not checked.

Recommendation:

Please make sure the bonding functions were successfully executed.

8.8.2. Multiple lines. Information: The **sellForETH** function uses absolute token balances instead of balance deltas, allowing pre-existing WETH or ETH held by the contract to be unintentionally transferred to users. Additionally, fee-on-transfer tokens may not be handled safely, because `amountIn` is passed as is without checking how many tokens were actually received.

Recommendation:

Please measure the actual token received or mention in a comment section why it is not necessary per business requirements.

```
uint256 wethBefore =
IERC20(weth).balanceOf(address(this));

bonding.sell(amountIn, tokenAddress, amountOutMin,
deadline);

uint256 wethAfter =
IERC20(weth).balanceOf(address(this));

uint256 wethReceived = wethAfter - wethBefore;
```

8.9. AeroAdaptor.sol:

8.9.1. Lines 33, 34, 35, 36. Information: variables router, tokenIn, tokenOut, factory could be marked "immutable".

9. Contracts Scoring legend:

SURGE score: 9

- 9.1. 9-10.** The highest-scoring contracts are ready for mainnet deployment. During the security audit auditors did not find any critical vulnerabilities or bugs that break contracts logic. Contracts gas-optimized.
- 9.2. 6-8.** The good-scoring results mean that auditors did not find many critical vulnerabilities or bugs that break contracts logic. Also, some parts of the contracts need to be rewritten or implemented (for instance, some parts of functionality are missing).
- 9.3. 1-5.** The low score means that contracts have many critical bugs, logic errors, vulnerabilities, are not ready for the mainnet, are not gas optimized, etc., and need to be rewritten from scratch.